

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Planning the Flutter Web–REST API–PostgreSQL Technology Stack of Your Own Project

Flutter Framework and Technology Stack

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 3
Output	Flutter Web–REST API–PostgreSQL technology stack blueprint, stack-role worksheets, and group presentation
Maximum Score	80 points

Student / Project Information

Block: <u>A-1</u>	Date: <u>9/17/2026</u>
System Title: SariwaBa?: Fish Freshness Classification System using Machine Learning and Mobile Computer Vision	Instructor: Jean Y. Villasin, MIT
Members: Marc Elson M. Rebato, Benedict A. Mendoza	

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 3

1. Activity Alignment and

Purpose

Topic Learning Outcome: Explain the prescribed Flutter Web–REST API–PostgreSQL technology stack and justify the role of each component in the proposed web system.

Course Outcome Alignment: CO1 – Explain web architecture and the roles of client-side, server-side, framework, API, database, and deployment components in the end-to-end flow of a web system.

Syllabus Assessment: Flutter–API–PostgreSQL Technology Stack Blueprint; stack-role explanation; online architecture justification task.

Purpose of the Activity

Activity 2 established the overall architecture of your semester project. In Activity 3, your team will convert that architecture into the prescribed implementation stack: Flutter Web for the client interface, a REST API using an instructor-approved backend framework/runtime for application logic, and PostgreSQL as the persistence layer developed in IT 108. You will also identify the role of Dart, Flutter packages, Git/GitHub, and the deployment environment so the blueprint can guide later implementation activities.

Important distinction: Flutter Web is the client application layer; the REST API is the application and integration layer; PostgreSQL is the persistence layer. The Flutter client must not contain database credentials or issue raw SQL directly to PostgreSQL in the prescribed stack.

2. Intended Learning Objectives

- Distinguish a framework, library/package, runtime, API style/protocol, DBMS, version-control tool, and deployment platform.
- Explain the responsibilities of Flutter Web, the REST API/backend, and PostgreSQL in the team's proposed system.
- Identify the Dart/Flutter fundamentals, widgets, packages, state-management approach, and client-side validation needed by major project features.
- Select and justify an instructor-approved backend framework/runtime for the REST API without shifting database or client responsibilities into the wrong layer.
- Map REST API resources and operations to the PostgreSQL tables, relationships, constraints, and transactions being developed in IT 108.
- Create a system-specific Flutter Web–REST API–PostgreSQL technology stack blueprint that includes Git/GitHub, security boundaries, and deployment targets.
- Explain and defend why each technology belongs in the stack and how the components communicate.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to the technology decisions and be able to explain one assigned stack component.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One project-specific Flutter Web–REST API–PostgreSQL Technology Stack Blueprint.

Activity Item	Requirement
Supporting Output	Technology classification worksheet, stack-role matrix, Flutter client plan, REST API plan, PostgreSQL mapping, Git/GitHub and deployment plan, 120–180 word stack-role explanation, and group presentation.
Recommended Tools	diagrams.net/Draw.io, Lucidchart, Figma, Canva, PowerPoint, Git/GitHub, Flutter/Dart documentation, backend API documentation, and PostgreSQL documentation.

Presentation Format	5–7 minutes per group followed by 2–3 minutes of questions. Every member must explain an assigned technology, interface, or integration decision.
----------------------------	---

4. Pre-Laboratory Requirements

- Final or revised architecture diagram from Laboratory Activity No. 2.
- Approved system title, primary users/roles, and at least five major system features.
- Current ERD or normalized PostgreSQL schema from IT 108, including primary keys, foreign keys, and important constraints.
- Instructor-approved backend framework/runtime for the REST API. If approval is still pending, bring no more than two candidate options for consultation.
- Initial list of Flutter packages/state-management tools and external services that the project genuinely needs.
- Access to the project GitHub repository and the team's planned deployment environment or hosting options.
- Copies of Flutter/Dart, REST API/backend, and PostgreSQL documentation relevant to the selected technologies.

5. Part A – Technology Classification and Role Check

Before assigning technologies to the project, classify each item correctly. Use the following distinctions: **framework** provides an application structure and conventions; **library/package** provides reusable functions or components; **runtime** executes application code; and the remaining categories describe communication, persistence, version control, or hosting responsibilities.

Category	Meaning
Framework	Provides structure, conventions, lifecycle, and reusable building blocks for an application.
Library / package	Reusable code added to the application for a focused capability such as HTTP, state management, charts, maps, or validation.
Runtime	Environment that executes application or server code.
API style / protocol / data format	Defines how components communicate and how requests/responses are represented.
DBMS	Manages persistent structured data, constraints, transactions, and queries.
Version control / collaboration	Tracks changes, supports branches and reviews, and stores project history.
Deployment platform / environment	Runs or hosts the client, API, database, and supporting services.

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 3 Classification Worksheet

Technology / Concept	Classification	Role in Your Project	Project-Specific Choice / Evidence
Dart	Programming Language	Implements Flutter client logic	Used with Flutter
Flutter	Framework	Builds the Web/Mobile presentation layer	Flutter Web Application and Flutter Mobile Application
Flutter package for HTTP/API calls	Library / Package	Sends REST requests to Django	Dio

State-management package or pattern	Library/package / state-management package	Manages scan/loading/result state	Riverpod State Management
REST + HTTPS + JSON	API style / protocol / data format	Defines how Flutter and Django communicate using REST endpoints over secure HTTPS with structured JSON responses	Flutter communicates with DRF endpoints
Python	Backend language	Implements backend logic and ML-related processing	REST API over HTTPS using JSON; image uploads use multipart/form-data where required
Backend runtime	Runtime Environment	Executes Django/Python server code	Python 3.x runtime
PostgreSQL	DBMS	Stores persistent application records	Users, classification_history, fish images
Git / GitHub	Version control / collaboration	Source tracking and team collaboration	Required project repository
Deployment platform(s)	Deployment platform/environment	Hosts Flutter Web, DRF, PostgreSQL, supporting services	Vercel for Flutter Web, Render Web Service for Django/ML, Render PostgreSQL for the database, and Android APK on user devices

6. Part B – Prescribed Stack Role Matrix

Complete the matrix using the exact technologies planned for your system. The “Must Not Be Responsible For” column is included to prevent duplicated logic and insecure shortcuts.

Layer / aArea	Prescribed Technology	Your Project-Specific		Must Not Be Responsible For
Presentation / Client	Flutter Web + Dart	Flutter Web + Dart	Provides SariwaBa user interfaces, navigation, image capture/upload, classification display, scan history, client-side validation, and user feedback	Database credentials, raw SQL, direct PostgreSQL access, final authorization decisions, or server-side business rules
Client packages	Team-selected Flutter packages	Dio, Riverpod, image/camera package, Google Sign-In package	Dio handles REST/HTTPS communication and multipart image uploads; Riverpod manages authentication, loading, classification, history, and UI state; image/camera packages support image acquisition; Google Sign-In handles client-side identity flow	Database administration, server secrets, ML inference, backend authorization rules, or database transactions
Integration	REST API over HTTPS using JSON	Django REST API accessed over HTTPS using JSON and multipart/form-data	Defines request/response communication, endpoint usage, HTTP methods, status codes, and data exchange between Flutter and Django	Persistent data storage, database constraints, or executing business rules by itself
Application / Backend	Backend framework/runtime	Django REST Framework + Python 3.x Runtime	Routes API requests; verifies identity and permissions; performs server-side validation; sanitizes uploads; preprocesses fish images; coordinates ML classification;	Rendering Flutter UI, storing production data in source-code files, exposing database credentials, or letting clients

			applies business rules; generates API responses; uses ORM for database access	bypass authorization
Persistence	PostgreSQL schema developed in IT 108	PostgreSQL - users, classification_history, fish images	Stores user profiles, scan/classification history, and image metadata/references; enforces keys, constraints, transactions, and relational integrity queries, durable records	UI logic, Flutter navigation, authentication screens, image classification, or client-state management
Version control	Git + GitHub	Git + GitHub	Stores source history; supports team collaboration, branches, commits, merges/pull requests, documentation, and project tracking	Running the deployed application or replacing deployment infrastructure
Deployment	Team-selected hosting / server / managed DB	Vercel + Render Web Service + Render PostgreSQL + Android APK	Hosts Flutter Web, Django/ML backend, managed PostgreSQL, environment configuration, and production deployment resources	Replacing application-layer security, database integrity rules, version control, or authorization logic

Stack-role rule

A good blueprint separates concerns. Flutter Web handles the user experience and client state; the REST API/backend protects and processes requests; PostgreSQL persists validated data and enforces relational integrity. Git/GitHub supports collaboration, while deployment platforms host the running components. If a responsibility appears in two layers, the team must explain why it is not duplicated or contradictory.

7. Part C – Flutter Web Client Blueprint

Plan the client side around your actual system features. List at least five important screens or feature modules. Use package names only when the team has decided to use them.

Feature / Screen	Widgets	State / Controller	Flutter Package(s)	API Operation Needed	Client Validation / User Feedback
1. Google Sign-In / Login Screen	Sign-in button, logo/title, loading indicator, error message	Riverpod authentication provider/controller	Riverpod; Google Sign-In package, Dio	POST /api/v1/auth/google/	Disable repeated sign-in while loading; show authentication error if login/token exchange fails
2. Home / Dashboard Screen	Navigation controls, scan/classify button, recent classification summary, user information/cards	Riverpod dashboard/navigation state	Riverpod, Dio	May request recent history/user session data	Show loading, empty, and error states; guide user to capture/upload an image
3. Fish Image Capture/Upload Screen	Camera/image preview, file picker, capture/upload button, classify button	Riverpod scan-session state	Riverpod; camera/file-selection package TBD	Prepares request for POST /api/v1/classify /	Accept approved formats such as .jpg and .png; check file-size bounds; show

					preview or validation error
4. Classification Processing/ Result Screen	Loading spinner, image preview, freshness badge, confidence score, retry/back button	Riverpod classification state	Riverpod; HTTP/API package TBD	POST /api/v1/classify/	Show processing/loading state; show Fresh / Not Fresh result and confidence; show retry/error feedback if request fails
5. Classification History Screen	List/table of past scans, date/time, freshness result, confidence, image reference	Riverpod history state	Riverpod; HTTP/API package TBD	GET /api/v1/history/	Show loading indicator, empty-history message, network/API error state
6. Classification Detail Screen	Fish image, freshness label, confidence score, scan metadata, back button	Riverpod selected-scan/detail state	Riverpod; HTTP/API package TBD	GET /api/v1/history/{id}/	Show loading/error state; prevent display of records the authenticated user is not allowed to access
7. Profile Screen	Name/profile information, account details, logout control	Riverpod authentication/profile state	flutter_riverpod, dio, Google Sign-In package	GET /api/v1/profile/, optional PATCH /api/v1/profile/	Validate editable fields if used; show save success/failure and logout feedback

Client planning checklist:

- Routing/navigation between screens is identified.
- Responsive web layout and loading/error/empty states are considered.
- Client-side validation improves usability but does not replace server-side validation.
- No database password, private API key, or other server secret is embedded in Flutter Web source code.

8. Part D – REST API and Backend Plan

Selected backend framework: Django REST Framework (DRF)

Selected backend runtime: Python 3.x Runtime

Why this choice fits the project (3–5 sentences):

SariwaBa uses Django REST Framework because it provides a centralized backend that can serve both Flutter Web and Flutter Mobile clients while keeping authentication, validation, business rules, and database access outside the presentation layer. DRF supports RESTful endpoints, structured JSON responses, permission checking, multipart image uploads, and secure integration with PostgreSQL through Django ORM. It also allows the application tier to coordinate Google identity verification and machine-learning classification without exposing backend or database credentials to the client. This separation supports maintainability, testing, security, and future scaling while remaining consistent with the selected three-tier architecture.

Minimum API Contract Worksheet

Define at least six REST operations that support the project's real workflows. Use an instructor-approved endpoint naming convention and appropriate HTTP methods.

Use Case	Method	Endpoint	Auth / Role	Request Data	Server Action / Rule	Response / Status
Google Authentication	POST	/api/v1/auth/google /	Public before verification	Google ID token	Validate token signature, issuer, audience, and expiry; identify or register corresponding user profile	200 OK with authenticated user/session data; 401 Unauthorized if token invalid
Classify Fish Image	POST	/api/v1/classify/	Authenticated Consumer/Vendor	Fish image via multipart/form-data + bearer token	Verify token and permissions; validate image; preprocess to RGB tensor; invoke ML classification; save classification record	200 OK JSON containing freshness label, confidence, and scan_id; 400 invalid image; 401 invalid auth
View Classification History	GET	/api/v1/history/	Authenticated Consumer/Vendor	Bearer token; optional query parameters if later supported	Return only classification records belonging to authenticated user	200 OK with JSON history list; 401 Unauthorized if unauthenticated
View Single Classification Record	GET	/api/v1/history/{id}/	Authenticated owner	Record ID in URL	Retrieve scan only if it belongs to authenticated user	200 OK with scan details; 403 Forbidden or 404 Not Found if inaccessible/not found
Delete Classification Record	DELETE	/api/v1/history/{id}/	Authenticated owner	Record ID in URL	Check record ownership before deleting related scan/history data according to DB rules	204 No Content; 403 Forbidden; 404 Not Found
Get User Profile	GET	/api/v1/profile/	Authenticated user	Bearer token	Retrieve user profile associated with verified identity	200 OK with user profile JSON; 401 Unauthorized
Update User Profile	PATCH	/api/v1/profile/	Authenticated user	Allowed profile fields only	Validate permitted fields and update authenticated user's profile	200 OK with updated profile; 400 Bad Request for invalid fields

REST API guidance

The API should validate input, verify identity and role permissions, apply business rules, interact with PostgreSQL or approved external services, and return an appropriate HTTP status with JSON data. Common methods include GET, POST, PUT/PATCH, and DELETE. The exact endpoints and rules must come from your own project requirements.

9. Part E – PostgreSQL Persistence and IT 108 Integration

Map the API resources to the PostgreSQL schema being designed in IT 108. Do not invent a second unrelated database model for IT 110. Record important keys, constraints, and transaction rules that affect the API.

API Resource / Feature	PostgreSQL Table(s)	PK / FK / Constraint	DML Operation	Transaction / Integrity Rule	Returned Fields / Result
Google-authenticated user profile	users	user_id as PK; Google identity value should be unique; required profile fields must not be null where applicable	SELECT, INSERT, possibly UPDATE	A Google identity should map to only one SariwaBa user record; creation/update must preserve unique identity mapping	user_id, display/profile information, account/session-related user data
Fish Freshness Classification	classification_history, fish_images, users	classification_history has its own PK; user reference is FK to users; image reference may link to fish_images	INSERT into classification/history and image metadata records	Classification record should only be saved for an authenticated user and after successful server-side validation/inference; related writes should be treated as one logical transaction where required	scan_id, label, confidence, image/reference metadata, timestamp
Classification History List	classification_history, optionally fish_images	History record belongs to a valid user through FK; only records linked to requesting user may be returned	SELECT	Backend must filter by authenticated user so one user cannot view another user's scan history	List of scan IDs, labels, confidence scores, timestamps, image references
Classification Detail	classification_history, fish_images, users	Record PK plus user FK; referenced image record must remain consistent with the history row	SELECT	Requested record must exist and belong to the authenticated user	Selected scan_id, freshness result, confidence, image metadata/reference, date/time
Delete Classification Record	classification_history, possibly fish_images	History PK; FK relationships determine whether related image metadata is restricted, cascaded, or deleted separately	DELETE	Ownership must be checked first; related records must remain referentially valid after deletion	Successful deletion result / no-content response
Retrieve User Profile	users	PK on user_id; unique Google identity mapping	SELECT	Only authenticated user's own profile should be returned	User profile fields
Update User Profile	users	PK preserved; unique and required-field constraints must remain valid	UPDATE	Only allowed profile attributes may be updated; identity keys should not be modified casually	Updated profile fields

Database boundary rule

In the prescribed Flutter Web–REST API–PostgreSQL stack, Flutter Web should not connect directly to PostgreSQL. The backend owns database credentials and uses the database driver/ORM or approved data access layer. PostgreSQL remains responsible for persistent records, keys, constraints, transactions, and query execution.

10. Part F – Git/GitHub and Deployment Plan

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Repository ownership	Location of Flutter client, backend, DB scripts/migrations, docs	Use a shared GitHub repository owned by the SariwaBa/DaisyMeRollin group. Both members should have contribution access. Repository visibility should follow instructor requirements.
Folder structure	Identify where Flutter client, backend API, documentation, and database scripts/migrations will be stored	Use one organized project repository with frontend/, backend/, database/, and docs/ directories
Branching / collaboration	Main branch, feature branches, commit/merge practice, and review responsibility	Keep main as the stable integration branch. Create feature branches such as feature/auth, feature/classification, feature/history, and feature/flutter-ui. Merge completed work through pull requests/review before integrating into main
Environment variables	List values that must be configured outside source code, such as API base URL and server/database secrets	Store Django secret key, database connection URL, Google client configuration required by the backend, deployment environment settings, allowed hosts, API base URL configuration, and similar secrets/configuration outside committed source files
Flutter Web deployment	Hosting platform or server for the built web client	Deploy the Flutter Web build to Vercel
REST API deployment	Server, container, cloud runtime, or approved hosting platform	Deploy Django REST Framework and the ML-related backend component to Render Web Service
PostgreSQL deployment	Local development instance and planned managed/server production database	Use local PostgreSQL during development; deploy production persistence using Render PostgreSQL
Logging / backup / recovery	Minimum plan for logs, error diagnosis, database backup, and restoration	Use Django/server logs for API errors and inference failures; use Render platform logs for deployment/runtime diagnosis; configure PostgreSQL backups/export procedures and document a restore process for recovery

Recommended minimum repository organization (adapt as needed):

```
SariwaBa/  
├── frontend/  
│   └── Flutter Web + Mobile  
├── backend/  
│   ├── manage.py  
│   ├── config/  
│   ├── api/  
│   ├── authentication/  
│   ├── classification/  
│   ├── history/  
│   ├── migrations/  
│   ├── requirements.txt  
│   └── .env.example  
├── docs/  
│   ├── architecture/  
│   └── technology-stack/  
├── README.md  
└── .gitignore
```

11. Part G – Construct the Technology Stack Blueprint

Create one clean, readable blueprint that shows how the prescribed stack is implemented in your own system. The blueprint must be understandable without opening the source code.

No.	Required Element	What Must Be Shown
1	System identity	Official project title and “Flutter Web–REST API–PostgreSQL” stack label.
2	Users / actors	Major users who interact with the Flutter Web client.
3	Flutter Web client	Dart, key screens/widgets, routing, state management, form validation, and HTTP/API client.

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 7 of 12
CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 3

No.	Required Element	What Must Be Shown
4	Flutter packages	Only packages actually planned for the project; state their purpose.
5	Communication	HTTPS, REST endpoints, JSON request/response, and API base URL/environment configuration.
6	Backend framework/runtime	Exact instructor-approved framework and runtime used to implement the API.
7	Application logic	Authentication, authorization, validation, services, business rules, and external-service integration.
8	PostgreSQL persistence	Relevant tables/data domains from IT 108, keys/constraints, and database access mechanism.
9	Git/GitHub	Repository and collaboration boundary for client, backend, database scripts, and documentation.

10	Deployment	Where the Flutter build, REST API, PostgreSQL database, and storage/external services will run.
11	Security boundary	Show where secrets live, where tokens/roles are checked, and which resources are protected.
12	Data flow	Directional arrows labeled with requests, responses, JSON, SQL/data access, and returned results.

Minimum Blueprint Rule

Your blueprint must show at least one complete implementation path: User → Flutter Web interface → HTTPS/JSON REST request → backend API logic → PostgreSQL transaction/query → JSON response → Flutter state/UI update. Include Git/GitHub and deployment information as supporting infrastructure, not as steps in the runtime request path.

12. Reference Example – Do Not Copy as Your Final Output

The figure below shows the level of stack separation and labeling expected. Replace every generic label with the technologies, features, endpoints, and database objects used by your own project.

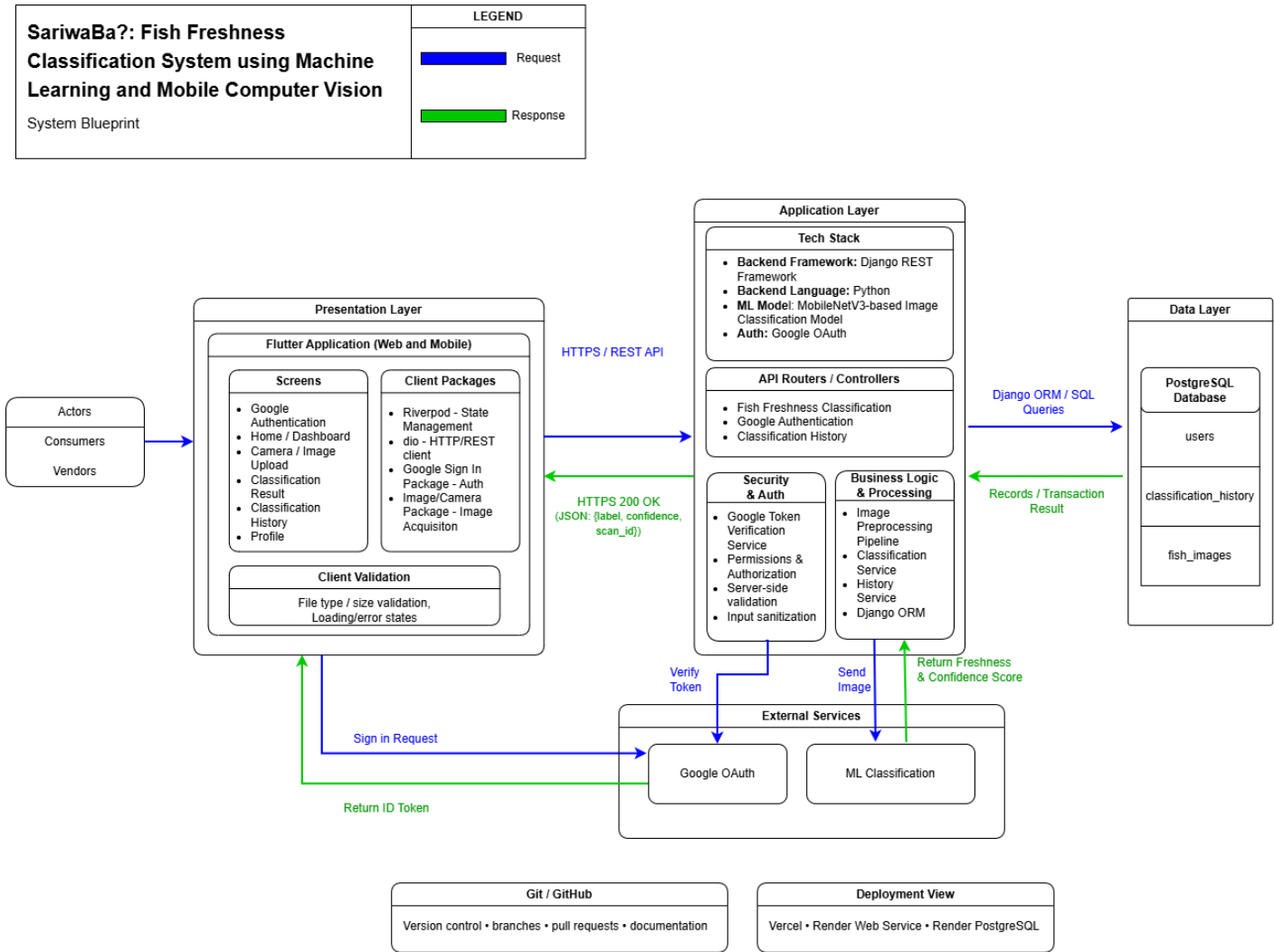


Figure 1. Reference Flutter Web–REST API–PostgreSQL technology stack and runtime path.

Step	Reference Flow
1	The Consumer or Vendor interacts with the Flutter Web/Mobile interface to sign in and capture or upload a fish image for freshness classification.
2	Flutter performs client-side image validation for file type and size, then Dio sends an authenticated HTTPS POST request to /api/v1/classify/ using multipart/form-data with the fish image and Google ID token.
3	The Django REST Framework API verifies the Google ID token and user permissions, performs server-side validation and input sanitization, preprocesses the image, and invokes the ML Classification Service to obtain the freshness verdict and confidence score.
4	After successful classification, Django uses the Django ORM to insert or update the corresponding records in PostgreSQL, including the authenticated user, classification history, and image metadata/reference while preserving transaction and relational-integrity rules.
5	Django returns an appropriate HTTP status and JSON response, such as 200 OK with {label, confidence, scan_id}.
6	Dio receives the response, Riverpod updates the classification state, and Flutter displays the Fresh / Not Fresh verdict, confidence score, and updated scan history to the user.

13. Part H – Feature-to-Stack Responsibility Trace

Selected workflow / feature: Fish Freshness Image Capture, Classification, and History Persistence

Expected result shown to the user: Flutter displays the fish freshness verdict (“Fresh” or “Not Fresh”), confidence score, and saves the classification as part of the authenticated user’s scan history.

Step	Responsible Component	Action / Data	Why It Belongs in This Component	Evidence in Blueprint
1	User – Consumer / Vendor	Captures or selects a fish image and starts the freshness classification process	The user initiates the workflow and provides the input that the system will process	Actors container: Consumers, Vendors
2	Flutter Web / Mobile – Presentation Layer	Displays image capture/upload interface and loads the selected image into the client	User interaction, image selection, and interface feedback belong in the presentation layer	Presentation Layer -> Flutter Application -> Screens -> Camera / Image Upload
3	Flutter Client Validation	Checks that an image is selected and validates supported file type and file-size limits	Client-side validation improves usability by detecting invalid input before network transmission	Presentation Layer -> Client Validation -> File type / size validation
4	Dio – Flutter Client Package	Constructs and sends POST /api/v1/classify/ over HTTPS using multipart/form-data, including the image and authentication token	Dio is the client-side HTTP package responsible for communicating with the REST API	Presentation Layer -> Client Packages -> dio – HTTP/REST client and HTTPS / REST API request arrow
5	Django REST	Verifies the Google ID	Authentication and authorization	Application Layer ->

	Framework – Security & Auth	token, checks permissions, and rejects unauthorized requests	must be enforced by the trusted backend rather than relying on Flutter	Security & Auth -> Google Token Verification Service; Permissions & Authorization
6	Django REST Framework – Server Validation	Performs server-side file/input validation and sanitization	Client validation cannot be trusted as the final security boundary, so the backend must validate the request again	Application Layer -> Security & Auth -> Server-side validation; Input sanitization
7	Django Business Logic & Processing	Decodes the image, resizes and normalizes it, and prepares a preprocessed RGB tensor	Image preprocessing is application/business logic required before the ML model can perform inference	Application Layer -> Business Logic & Processing -> Image Preprocessing Pipeline
8	ML Classification Service	Receives the preprocessed RGB tensor, performs computer-vision inference, and returns the freshness label and confidence score	ML inference is a specialized supporting service and is separated from UI and persistence responsibilities	External Services -> ML Classification
9	Django Business Logic & Processing	Receives the ML result, formats the freshness verdict/confidence, and prepares classification/history data for persistence	The backend coordinates the external ML result with the system's business rules and persistence workflow	Application Layer -> Business Logic & Processing -> Classification Service; History Service
10	Django ORM + PostgreSQL	Django ORM inserts the classification/history and image metadata/reference into PostgreSQL	Database access must be performed by the backend; PostgreSQL is responsible for durable storage, transactions, and relational integrity	Business Logic & Processing -> Django ORM and Data Layer -> PostgreSQL Database → users, classification_history, fish_images
11	Django REST Framework	Returns an HTTP status and JSON payload such as {label, confidence, scan_id}	The backend is responsible for generating the API response after processing and persistence succeed	HTTPS 200 OK (JSON: {label, confidence, scan_id}) response arrow
12	Dio + Riverpod - Presentation Layer	Dio receives the response; Riverpod updates classification/history state	Dio handles network response delivery while Riverpod manages client application state	Client Packages -> dio – HTTP/REST client; Riverpod – State Management
13	Flutter Web / Mobile UI	Displays the Fresh/Not Fresh result, confidence score, and updated history state to the user	Rendering the result and user feedback are presentation-layer responsibilities	Screens -> Classification Result; Classification History

14. Part I – Stack-Role Explanation

Write 120–180 words explaining and justifying the prescribed technology stack for your own system. Address all of the following:

- What Flutter Web and Dart are responsible for in the user experience.
- Why the REST API/backend is the correct place for authentication/authorization checks, server-side validation, and business rules.
- How PostgreSQL persists the data model being developed in IT 108 and protects relational integrity.
- Why

Git/GitHub and the deployment environment are necessary even though they are not runtime application layers. • How the separation improves security, maintainability, testing, integration, and future scaling, including one implementation risk the team must manage.

SariwaBa uses a Flutter Web–REST API–PostgreSQL technology stack to keep client, application, and persistence responsibilities clearly separated. Flutter Web with Dart handles the user interface, image capture/upload workflow, client-side validation, navigation, and state updates through Riverpod, while Dio manages HTTPS requests to the backend. Django REST Framework is responsible for authentication and authorization checks, server-side validation, business rules, image preprocessing, ML-service coordination, and secure PostgreSQL access through Django ORM. PostgreSQL persists user, classification-history, and image-reference data while enforcing transactions and relational integrity. Git and GitHub support version control, collaboration, documentation, and controlled code changes, while Vercel, Render Web Service, and Render PostgreSQL provide the deployment environment even though they are not runtime application layers. This separation improves security, maintainability, testing, integration, and future scaling. One implementation risk is ML inference and image-upload latency, which the team must manage through efficient preprocessing, reasonable file-size limits, and responsive loading/error states.

15. Final Quality Checklist

Checklist A	Checklist B
☐ The official system title and actual project features are used.	☐ The stack is explicitly labeled Flutter Web–REST API–PostgreSQL.
☐ Framework, library/package, runtime, DBMS, version control, and deployment platform are classified correctly.	☐ Flutter screens/widgets, client state, packages, and API calls are project-specific.
☐ The instructor-approved backend framework/runtime is named.	☐ At least six meaningful REST operations are defined.
☐ Authentication, authorization, validation, and business rules are located primarily in the backend/API layer.	☐ The PostgreSQL mapping matches the IT 108 schema or clearly identifies coordinated schema changes.
☐ Flutter Web does not contain database credentials or direct raw SQL access to PostgreSQL.	☐ HTTPS/REST/JSON and database-access arrows are directional and meaningfully labeled.
☐ Git/GitHub organization, environment variables, and deployment targets are identified.	☐ At least one complete User → Flutter → API → PostgreSQL → API → Flutter path is visible.
☐ The written stack-role explanation agrees with the blueprint and worksheets.	☐ Every group member can explain one component and defend the complete integration path without reading a script.

16. Submission Requirements

- PDF or PNG of the final technology stack blueprint in readable landscape orientation.
- Editable blueprint source file such as .drawio, .pptx, .fig, or the native file of the selected diagramming tool.
- Completed Parts A–H and the 120–180 word stack-role explanation.
- Updated project GitHub repository containing the blueprint and supporting files under a folder such as docs/technology-stack/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA3_SystemName.
- Deliver a 5–7 minute group presentation using the final blueprint as the main visual, followed by 2–3 minutes of questions. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own system and technology decisions. AI tools may be used to clarify terminology or review ideas, but the final stack classification, framework/runtime selection, endpoint plan, PostgreSQL mapping, blueprint, and justification must be verified, understood, and defended by the team. The instructor may ask any member to explain why a component belongs in a particular layer or trace a request without assistance.

17. Scoring Rubric – 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. For the presentation criterion, points may be reduced for unequal participation, excessive reading, incorrect role assignment, weak integration reasoning, or inability to answer questions about the team's own stack.

Criterion	Evidence for Full Credit	Points
-----------	--------------------------	--------

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 10 of 12
CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 3

Criterion	Evidence for Full Credit	Points
Framework/library/runtime and stack-role accuracy	Technologies are classified correctly, and the team clearly distinguishes Flutter, Dart, packages, REST, backend runtime/framework, PostgreSQL, Git/GitHub, and deployment responsibilities.	10
Flutter Web client planning	Screens/features, widgets/forms, state approach, packages, client validation, API calls, loading/error feedback, and client security boundaries are complete and project-specific.	10
REST API and backend plan	An instructor-approved backend framework/runtime is justified; endpoints, methods, authentication/roles, validation, business rules, request data, responses, and status behavior are coherent.	14
PostgreSQL persistence integration	API resources map correctly to the IT 108 schema; important tables, PK/FK constraints, DML operations, transactions, and integrity requirements are represented accurately.	11
Technology stack blueprint and end-to-end integration	The blueprint clearly integrates users, Flutter Web, packages/state, HTTPS/REST/JSON, backend application logic, PostgreSQL, security, and a complete request-response path.	15
Git/GitHub, deployment, and secret management	Repository organization, collaboration approach, environment variables, hosting targets, and protection of server/database secrets are identified and realistic.	5

Written stack-role explanation	The 120–180 word explanation accurately justifies component roles, separation of concerns, IT 108 integration, security, maintainability, testing, deployment, scaling, and one implementation risk.	5
Group presentation and oral defense	Presentation is clear, organized, within time, and uses the final blueprint effectively; all members participate and correctly defend technology roles, API/database boundaries, and the selected workflow.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which responsibility was hardest for your group to assign correctly among Flutter Web, the REST API, and PostgreSQL, and why?

The hardest part was separating validation between Flutter and the REST API because both perform validation, but the backend must still do the final security and business-rule checks.

Which Flutter package or client-side pattern is most important to your project, and what problem will it solve?

Riverpod is the most important because it manages loading, authentication, classification results, history, and other UI states in an organized way.

What server-side rule must never rely only on Flutter client validation?

Image validation and user authorization must never rely only on Flutter because client-side checks can be bypassed.

Which PostgreSQL constraint or transaction rule from IT 108 is most important to preserving correct system data?

Foreign key constraints are important because they keep classification records correctly linked to valid users and related data

What must your team configure before the first real Flutter-to-REST-API integration test can succeed?

We must configure the API base URL, Django server settings, CORS, required environment variables, and the connection between Flutter and the REST API.

What is the first stack component your group should implement after this activity, and what evidence will prove that it works?

We should first implement the basic Flutter-to-Django image upload endpoint. It works if Flutter can send an image and Django successfully receives it and returns a valid JSON response.

